

Governance charter

Petappro is a white-label SaaS platform spanning web and native mobile. To stay coherent for years — not just to MVP — every component earns its place. This charter is the standard: nothing enters the library until it passes. The goal is a system that **strengthens** with each addition instead of accruing design debt.

Principle. Treat the design system as a product in its own right. Composition over duplication; reusable over page-specific; semantic tokens over literals; documented over implied.

01

Roles

Three complementary roles, one pipeline. Clear ownership keeps velocity high and the library clean.

OWNS VISION

Product Design Director

You. Owns product vision, UX strategy, brand, and the user experience. Sets priorities and makes the final call.

PRODUCES

Production Designer

Claude. Builds components, pages, and layouts in Figma to spec — fast iteration, faithful to tokens and rules.

GOVERNS

Design System Governor

George. Ensures every new component strengthens the system: owns governance, implementation guidance, architecture, and quality control — running the acceptance checklist and standing review before anything is merged.

02

System layers

Five layers. The fourth — **Domain System** — is what makes this Petappro's system and not a generic kit.

1 • Foundations

Tokens: color (semantic, themed via modes), type, spacing, radius, elevation, motion, breakpoints, grid.

2 • Atoms & components

Button, Input, Avatar, Icon, Money, Badge, Switch, Chip — generic, reusable primitives.

3 • Patterns

Section Header, Date Navigator, field groups, list rows, pricing rows — recurring compositions.

4 • Domain system • pet-care first-class

Components that exist only because Petappro is a pet-care platform. They encapsulate visual treatment *and* business semantics. (See §05.)

5 • Templates

Full screens: Dashboard, Booking, Profile, Schedule, Reports — composed only from the layers above.

03

Acceptance checklist

Every component passes all five groups before it joins the library. The Governor signs off.

1 Foundations

- ✓ Uses semantic tokens, never hard-coded values.
- ✓ Follows the spacing & sizing scales (4px grid).
- ✓ Meets accessibility requirements (WCAG 2.1 AA).
- ✓ Theme-ready across all 5 modes + custom white-label. Dark mode deferred by decision; structure supports it later.
- ✓ Uses approved typography tokens.

2 Component quality

- ✓ Correct Auto Layout (hug/fill deliberate).
- ✓ Logical variants — no redundant ones.
- ✓ Minimal, well-named properties.
- ✓ Consistent naming convention.
- ✓ Correct nested-component usage.
- ✓ Proper constraints & resizing behavior.

3 UX quality

- ✓ Complete interaction states (default, hover, pressed, focus-visible, disabled).
- ✓ Loading and empty states.
- ✓ Error & validation states.
- ✓ Keyboard / focus behavior.
- ✓ Mobile considerations.
- ✓ Touch targets ≥ 44px.

4 Engineering readiness

- ✓ Maps cleanly to React Native components.
- ✓ Avoids unnecessary complexity.
- ✓ Reusable rather than page-specific.
- ✓ Composition over duplication.
- ✓ Supports future white-label branding.

5 Documentation

- ✓ Purpose.
- ✓ Usage.
- ✓ When not to use it.
- ✓ Variant descriptions.
- ✓ Accessibility notes.
- ✓ Developer implementation notes.

04

Prompts as specifications

We write specs, not requests. Every component prompt follows **Objective → Requirements → Acceptance Criteria**, so what Claude produces is verifiable against a contract — the same artifact a mature design-systems team would maintain.

SPEC TEMPLATE Canonical component prompt

Copy

OBJECTIVE

Create the canonical <Component> for the Petappro Design System.

REQUIREMENTS

- Semantic variants: ...
- Sizes: ...
- States: default, hover, pressed, focus-visible, disabled, loading/empty where relevant
- Tokens: colors from theme roles; spacing from scale; radius from comp.<component>.shape
- Accessibility: WCAG 2.1 AA – contrast, focus ring ≥3:1, touch target ≥44px
- Auto Layout: direction, hug/fill, gap = space token
- Slots: named content regions (leading/trailing/body ...)
- React Native compatibility: property + variant names map to RN prop names

ACCEPTANCE CRITERIA

- No hard-coded colors or literal hex.
- No detached instances.
- All variants use shared tokens.
- Meets WCAG AA.
- Uses component properties appropriately (minimal, well-named).
- Reusable, not page-specific; composition over duplication.

DOCUMENTATION

- Purpose · Usage · When not to use · Variant descriptions · Accessibility notes · Dev notes.

Note: Part 2 of the Build Playbook will render every atom, pattern, and domain component in this exact spec format. The example Button prompt has already been upgraded to match.

05

Domain system

First-class citizens, not ad-hoc screen UI. Each encapsulates a pet-care concept — visual treatment plus business semantics — grounded in your live app's taxonomy.

Booking Card

Reservation summary: service, dates, pets, price, status.

Pet Avatar

Photo/fallback, gender-tinted ring, puppy indicator.

Multi-pet Avatar Stack

Overlapping avatars + overflow count for a household.

Boarding Status Badge

Upcoming · In-progress · Completed · Cancelled.

Vaccination Badge

Current / expiring / missing with date semantics.

Temperament Tag

Behavior labels (friendly, reactive, shy...).

Medication Chip

Med + dose + schedule for a stay.

Service Pill

Boarding · Daycare · Walk · Grooming · Training · M&G.

Booking Timeline

Arrival → stay → departure milestones.

Calendar Availability Block

Open / limited / blocked capacity per day.

Capacity Meter

Occupancy vs. limit, with threshold colors.

Client Risk Indicator

Outstanding balance, incomplete M&G, flags.

Payment Status Badge

Paid · Unpaid · Partial · Overridden.

Report Card Component

Dashboard/report metric tile with delta + drill-in.

06

Standing review

Every new component or pattern is reviewed against four questions. If any answer is “no,” we refine before it becomes part of the system.

1 Is this visually consistent?

2 Is this scalable?

3 Is this implementable without unnecessary engineering complexity?

4 Six months from now, will we be glad we built it this way?

07

Component record & versioning

We treat the design system as its own product. Every component carries a **record** — so engineering can trust it and it can evolve without becoming chaotic. No record, not in the library.

Every component ships with

- ✓ A clear **purpose** — one line on what it's for.
- ✓ **Acceptance criteria** — the contract it was built against (§04).
- ✓ A **governance review** — checklist + four-question sign-off, dated, by George.
- ✓ An **implementation-ready spec** — tokens, props, states, RN mapping notes.
- ✓ A **version history** — changelog for any significant change.

Versioning rule

Semantic-style versions per component and for the system as a whole:

MAJOR · x.0.0

Breaking change — renamed/removed props or variants; consumers must update.

MINOR · 0.x.0

Additive — new variant/state/prop, backward compatible.

PATCH · 0.0.x

Fixes — spacing, color token, a11y tweak; no API change.

Where it lives. Each Figma component gets a record frame beside it (purpose · criteria · review date · spec · changelog). The system keeps a top-level CHANGELOG on the Cover page. Version bumps are decided at governance

